Instrukcja laboratoryjna 1

Wprowadzenie do wielozadaniowości w systemach czasu rzeczywistego (FreeRTOS, Arduino)

Środowisko: Arduino IDE lub symulator Wokwi, płytka Arduino UNO (lub ESP32), nakładka multifunkcja (opcjonalnie diody LED z rezystorami, przycisk).
Związek z wykładem: wykład 2 — zadania, stany zadań, planista, tick, vTaskDelay() i vTaskDelayUntil().

> **Ważne dla ESP32:** układ ma **dwa rdzenie**. Funkcja xTaskCreate() nie przypina zadania do rdzenia, więc dwa zadania mogą pracować jednocześnie na różnych rdzeniach — wtedy wpływ priorytetów jest słabo widoczny. W zadaniach 3 i 5 wszystkie zadania tworzymy funkcją **xTaskCreatePinnedToCore(..., 1)** (rdzeń 1), żeby zobaczyć działanie planisty tak jak na procesorze jednordzeniowym. Czasy zawsze zapisujemy przez **pdMS_TO_TICKS(ms)**.

Zadanie 1: Sekwencyjne miganie diodami

1. Utwórz projekt w dla Arduino UNO lub ESP32 z dwiema diodami LED (z rezystorami).
2. W funkcji loop() zaimplementuj miganie diody 1 przez 500 ms, a następnie diody 2 przez 1000 ms — sekwencyjnie, z użyciem delay().
3. Zaobserwuj, że nie ma prawdziwej „równoczesności" — jedna dioda miga dopiero po zakończeniu pracy drugiej.

Do analizy: ile wynosi czas reakcji programu na zdarzenie (np. naciśnięcie przycisku sprawdzane raz na obieg pętli) w najgorszym przypadku?

Zadanie 2: Prosta wielozadaniowość

1. W funkcji setup() utwórz funkcją xTaskCreate() dwa zadania:
   - zadanie A: miga diodą 1 co 500 ms,
   - zadanie B: miga diodą 2 co 1000 ms.
2. W zadaniach użyj vTaskDelay(pdMS_TO_TICKS(...)) zamiast delay().
3. Uruchom projekt — obie diody migają niezależnie. Zwróć uwagę, że funkcja loop() jest pusta.

```
void TaskA(void *pv) {
  pinMode(LED1, OUTPUT);
  for (;;) {
    digitalWrite(LED1, !digitalRead(LED1));
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}


void setup() {
  xTaskCreate(TaskA, "A", 2048, NULL, 1, NULL);   // stos w bajtach (ESP32)
  xTaskCreate(TaskB, "B", 2048, NULL, 1, NULL);
}
void loop() {}
```

> **Uwaga:** na ESP32 rozmiar stosu w xTaskCreate() podaje się w **bajtach** (w standardowym FreeRTOS — w słowach).

Zadanie 3: Kolejność wykonywania zadań i priorytety

Etap A — zadania, które się blokują

1. Utwórz 3 zadania A, B i C (w ESP32 przypięte do rdzenia 1). Każde co sekundę wypisuje swój identyfikator (Serial.println("A") itd.).
2. Ustaw priorytety: A — 1, B — 2, C — 3.
3. Obserwuj kolejność komunikatów w konsoli. Następnie zmień priorytety i porównaj.

Etap B — zadanie, które się nie blokuje

4. Usuń vTaskDelay() z zadania C (priorytet 3) — niech wypisuje komunikat w pętli bez przerwy.
5. Obserwuj, czy zadania A i B nadal działają. Odczytaj komunikaty systemowe w konsoli (watchdog zadań).
6. Przywróć opóźnienie w zadaniu C, a zamiast tego obniż jego priorytet do 1 (taki sam jak A). Opisz zmianę zachowania.

Do analizy:

- Dlaczego w etapie A priorytety wpływają tylko na kolejność komunikatów, a nie na ich liczbę?
- Na czym polega głodzenie (starvation) zadań i dlaczego zgłasza się watchdog?
- Co zmienia równy priorytet (time slicing)?

Zadanie 4: Kontrola cyklu życia zadania

1. Utwórz zadanie, które miga diodą 3 razy i usuwa się samo: vTaskDelete(NULL).
2. W funkcji loop() (lub w osobnym zadaniu nadzorczym) co 5 s sprawdzaj, czy zadanie istnieje, i twórz je ponownie.

```
TaskHandle_t hBlink = NULL;


void Blink3(void *pv) {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED1, HIGH); vTaskDelay(pdMS_TO_TICKS(200));
    digitalWrite(LED1, LOW);  vTaskDelay(pdMS_TO_TICKS(200));
  }
  hBlink = NULL;              // informacja dla nadzorcy
  vTaskDelete(NULL);          // zadanie usuwa się samo
}


void loop() {                 // nadzorca
  if (hBlink == NULL) xTaskCreate(Blink3, "blink3", 2048, NULL, 1, &hBlink);
  vTaskDelay(pdMS_TO_TICKS(5000));
}
```

Do analizy: w jakich stanach (Running, Ready, Blocked, Suspended, Deleted) znajduje się zadanie Blink3 w trakcie swojego życia?

Zadanie 5: Różne czasy pracy zadań (obciążenie procesora)

1. Utwórz dwa zadania (w ESP32 przypięte do rdzenia 1):

- zadanie A: miga diodą co 500 ms,
- zadanie B: wykonuje serię „ciężkich" obliczeń (np. pętla for trwająca ok. 300 ms), po każdej serii wypisuje komunikat i czeka 100 ms.

2. Ustaw ten sam priorytet dla obu zadań. Uruchom i zaobserwuj miganie diody oraz tempo komunikatów zadania B.
3. Zmień priorytet tylko zadania B na wyższy. Uruchom ponownie i porównaj zachowanie.
4. Usuń 100 ms oczekiwania z zadania B i ponownie porównaj.

Do analizy: jak planista dzieli czas procesora przy równych priorytetach, a jak przy różnych? Kiedy dioda przestaje migać równo?

Zadanie 6: vTaskDelay() a vTaskDelayUntil() — pomiar okresu

1. Utwórz zadanie, które w każdym obiegu wykonuje „pracę" trwającą ok. 20 ms (pętla aktywnego oczekiwania), a następnie czeka.
2. Wariant 1: oczekiwanie vTaskDelay(pdMS_TO_TICKS(100)). Wariant 2: vTaskDelayUntil(&last, pdMS_TO_TICKS(100)).
3. W obu wariantach zmierz rzeczywisty okres (w ESP32 możesz zrobić to funkcją esp_timer_get_time())dla 50 obiegów: wyznacz średni okres, minimum i maksimum.
4. Wypisz zapas stosu zadania funkcją uxTaskGetStackHighWaterMark(NULL).

```
void Okresowe(void *pv) {
  TickType_t last = xTaskGetTickCount();
  int64_t prev = esp_timer_get_time();
  for (int i = 0; i < 50; i++) {
    int64_t t0 = esp_timer_get_time();
    while (esp_timer_get_time() - t0 < 20000) { }      // praca 20 ms
    vTaskDelayUntil(&last, pdMS_TO_TICKS(100));        // wariant 2
    int64_t now = esp_timer_get_time();
    Serial.println((long)(now - prev));                // okres [us]
    prev = now;
  }
  Serial.printf("wolny stos: %u B\n", uxTaskGetStackHighWaterMark(NULL));
  vTaskDelete(NULL);
}
```

Do analizy: porównaj zmierzone okresy z przewidywaniami z wykładu 2 (okres = praca + opóźnienie albo stały okres). Ile obiegów wykona każdy wariant w ciągu minuty?

Sprawozdanie

- Kody programów z zadań 2–6 (fragmenty istotne dla zadania) i opis obserwacji.
- Odpowiedzi na pytania „Do analizy".
- Tabela wyników zadania 6: średni, minimalny i maksymalny okres dla obu wariantów oraz wniosek.